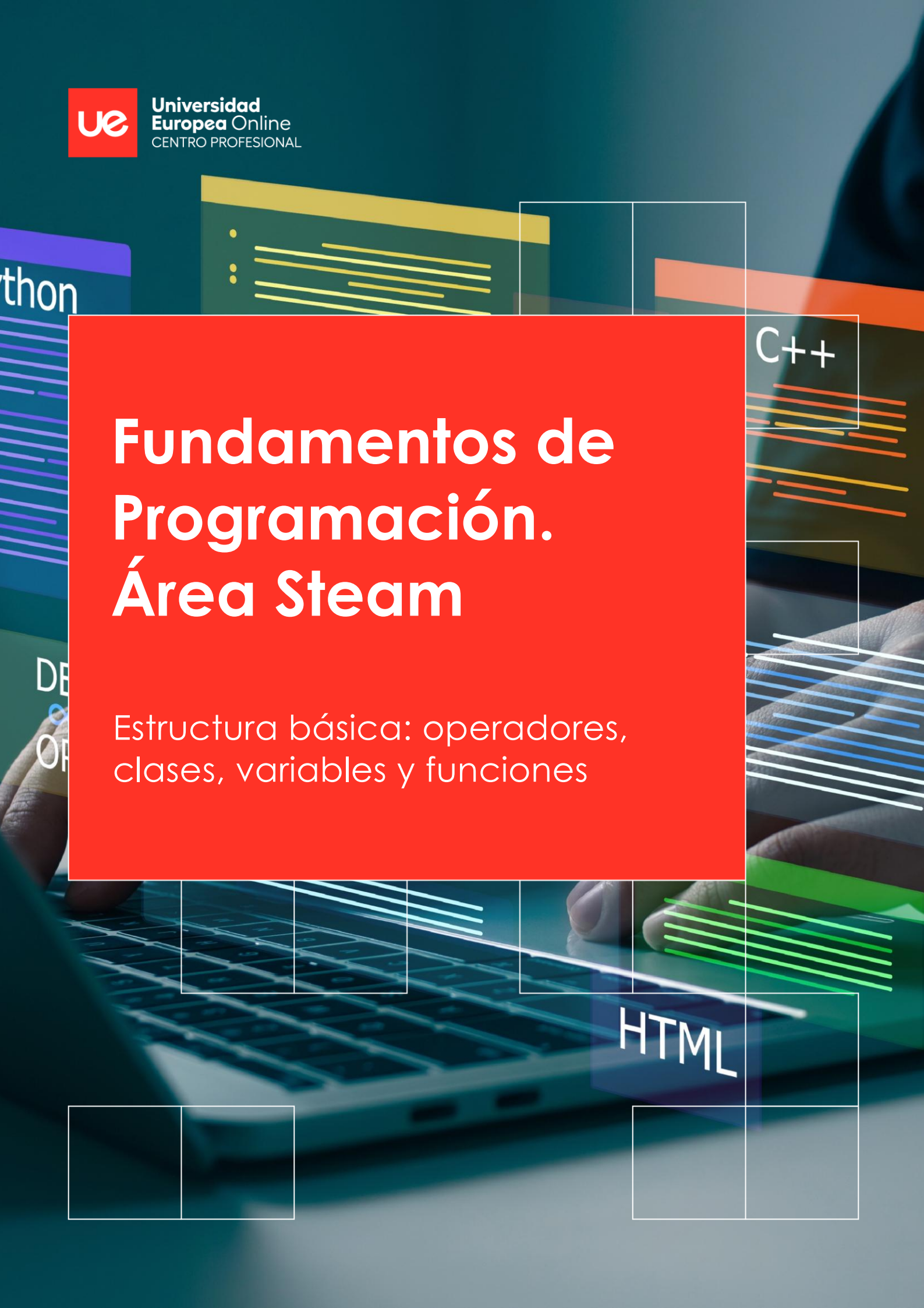
Universidad Europea Online
CENTRO PROFESIONAL



Fundamentos de Programación. Área Steam

Estructura básica: operadores, clases, variables y funciones

Índice

Presentación 3
Objetivos de aprendizaje 3
1. Estructura de un programa informático 3
2. Palabras reservadas 4
3. Componentes de un programa 5
3.1. Clases y *namespace* 6
3.2. Variables 7
3.3. Métodos 10
4. Operadores 13
Resumen 16
Referencias bibliográficas 16

© Centro de Formación Profesional Europeo de Madrid

Presentación

Una vez se ha instalado y configurado todo el entorno de desarrollo, es momento de ponerse manos a la obra. Para ello, es fundamental entender los componentes principales que todo programa debe tener, y que permiten agregar la funcionalidad necesaria para que se lleven a cabo las tareas que solventan el problema planteado. Para ello, además de aprender la estructura y elementos de un programa, es necesario conocer los diferentes tipos de datos que están a nuestra disposición en el código, para así poder utilizar el que mejor se adapte a nuestras necesidades.

A lo largo de este tema veremos cómo poder empezar a programar nuestros propios algoritmos utilizando clases, las variables y los métodos. Además, para poder utilizar y gestionar las acciones que se quieren realizar en cada uno de los métodos, utilizaremos los diferentes tipos de operadores que existen para realizar todo tipo de tarea que sea requerida, todo ello envuelto en el marco de desarrollo C#.

Objetivos de aprendizaje

Los objetivos que se pretenden alcanzar en este recurso son los siguientes:

- Conocer cada uno de los elementos de un programa.
- Saber las palabras reservadas del lenguaje C#.
- Conocer y utilizar los diferentes tipos de datos disponibles.
- Entender y comprender el uso de las clases y los *namespace*.
- Entender el concepto de *variable* y sus tipos.
- Entender el concepto de *método*, *parámetros* y *retornos*.
- Entender el concepto de *operador* y los diferentes tipos.

1. Estructura de un programa informático

Un programa informático es un conjunto de instrucciones que permiten el tratamiento de datos cuyo origen puede ser diverso. Dentro de un programa informático desarrollado con C#, nos encontraremos con una serie de ficheros con la extensión .cs que representan cada una de las clases, donde estarán los códigos que la máquina ejecutará. Estos podrán tener diversos tipos de elementos dentro como *namespace*, clases, variables, funciones, estructuras de control, etc., teniendo cada uno de ellos una funcionalidad diferente que iremos desgranando a lo largo del tema.

Antes de empezar a definir cada uno de ellos, podemos decir, de forma general, que la estructura de un programa informático (de cada uno de los archivos .cs que lo forma) es la siguiente:

© Centro de Formación Profesional Europeo de Madrid

Cabeceras	Comentarios: donde se indican el nombre de la clase, datos que gestiona, autor, etc.
Declaraciones	Variables: definición de cada una de las variables que manejará el programa.
Funcionalidad	Funciones o métodos: definición de todos aquellos métodos que se pueden ejecutar en cualquier fichero.
Estructuras	Dentro de las funciones o métodos, encontraremos el uso de estructuras de control y operadores para poder darle lógica a cada una de las acciones del programa.

Importante

Como se verá a continuación, estos elementos están englobados en lo que se conoce como clases, que a su vez están dentro de los conocidos como *namespace*. Esta estructura es básica para el correcto funcionamiento de los programas y el acceso seguro a cada una de las funcionalidades.

2. Palabras reservadas

Como en todo lenguaje de programación, en C# hay determinadas palabras que no se pueden utilizar, salvo para indicar determinadas acciones o elementos que vienen marcadas por el propio lenguaje. Estas palabras, se puede decir que son identificadores predefinidos que tienen un significado ya marcado para el compilador, y no podrán ser utilizadas para ningún otro fin dentro del código. Por ejemplo, la palabra *if* representa una palabra reservada, ya que indica el comiendo de una estructura de control, por lo que no podremos crear una variable con dicho nombre, ya que de ser así el compilador lo interpretará como el comienzo de una estructura.

Existen multitud de palabras reservadas, casi todas ellas se pueden identificar como: estructuras de control, modificadores de acceso, tipo de datos, tipo de variable / retorno o modificadores que hacen referencia a como se trata una función o clase. Alguna de estas palabras reservadas que solo podrán ser utilizas con el fin que tienen predefinido son:

Palabras reservadas				
abstract	*delegate*	*if*	*override*	*struct*
as	*do*	*implicit*	*params*	*switch*
base	*double*	*in*	*private*	*this*

© Centro de Formación Profesional Europeo de Madrid

bool	*else*	*int*	*protected*	*throw*
break	*enum*	*interface*	*public*	*true*
byte	*event*	*internal*	*readonly*	*try*
case	*explicit*	*is*	*ref*	*typeof*
catch	*extern*	*lock*	*return*	*uint*
char	*false*	*long*	*sbyte*	*ulong*
checked	*finally*	*namespace*	*sealed*	*unchecked*
class	*fixed*	*new*	*short*	*unsafe*
const	*float*	*null*	*sizeof*	*ushort*
continue	*for*	*object*	*stackalloc*	*using*
decimal	*foreach*	*operator*	*static*	
default	*goto*	*out*	*string*	

Como se verá en los siguientes puntos, cada una de estas palabras guarda un significado en C# y son de muy fácil identificación ya que al escribirlas se autocompletará el código y además cambiará el color de la palabra para así saber que estamos tratando con una palabra reservada.

Importante

El pseudocódigo es un lenguaje intermedio entre el lenguaje natural y el lenguaje de programación de muy fácil compresión y no ejecutable, que permite comprender la lógica de la solución de forma sencilla.

3. Componentes de un programa

Antes de empezar a definir cada uno de los elementos que forman un programa informático, es importante remarcar una serie de aspectos que ayudarán mucho a la hora de crear cada una de las partes de nuestro programa. No se trata de ningún elemento escribo ni que haya que aprenderse de memoria, pero sí una guía de buenas prácticas a la hora de programar. Son los siguientes:

1. Sé ordenado. Esto es clave para poder escribir un código que sea compresible. Cuando programamos debemos tener la capacidad de crear un código lo más limpio posible para que nosotros mismos seamos capaces de entenderlo.
2. Soluciona un problema para pasar al siguiente. No dejes las cosas a medias, ya que cada uno de los elementos del programa están relacionados.
3. Intenta estructurar el código lo mejor posible, guardando los espacios y justificaciones necesarias. Esto te ayudará a entender visualmente el código.

© Centro de Formación Profesional Europeo de Madrid

Todos los IDE tienen un atajo de teclado para formatear el código. Este lo dejará ordenado y mucho más comprensible.

Un programa informático se compone principalmente *namespace*, clases, variables y métodos. Todos ellos guardan una relación directa entre cada uno de ellos, y aunque no todos son obligatorios, en la mayoría de los casos nos los encontraremos.

Para poder utilizarlos, lo primero y más importante es tener en cuenta que necesitamos un archivo .cs con formato *PascalCase*, lo que indica que la primera letra es mayúscula, todo junto y las siguientes palabras la primera también en mayúscula. Este nombre debe coincidir con el nombre de la clase principal que se escriba dentro del archivo. Este tipo de nomenclatura servirá también para el nombre de los *namespace*, clase y métodos. Una vez creado el archivo se puede empezar a definir cada uno de los elementos

En programación existe otro tipo de nomenclatura además del *PascalCase*, que es la llamada *camelCase*. Esta indica que la primera letra es minúscula. Dependiendo del estándar de codificación que utilice el lenguaje de programación se utilizará una u otra. Podéis encontrar una guía de estilo de C# en el siguiente enlace:

<https://google.github.io/styleguide/csharp-style.html>.

3.1. Clases y *namespace*

Son los elementos principales de los archivos escritos en C#. Un *namespace* se utiliza para declarar el ámbito que compartirán todos los elementos declarados dentro de este. Para su declaración se utiliza la palabra reservada *namespace*.

```
namespace EjemploNamespace

{

  public class Program {


  }


  public class OtherClass {


  }

}
```

En este ejemplo se puede ver como las dos clases declaradas dentro del *namespace* comparte un mismo ámbito, por lo que podrán compartir diferentes elementos entre sí. Además de los *namespace* que nosotros podamos crear como el del ejemplo, también

© Centro de Formación Profesional Europeo de Madrid

podemos utilizar todos aquellos que ya están creados, aprovechando así su funcionalidad.

Importante

Texto Dentro de un *namespace* tan solo se puede declarar de forma directa clases, interfaces, enum, métodos delegados u otros *namespace*, pero nunca métodos. Además, todo el contenido de este está englobado entre llaves, al igual que el resto de los elementos.

A diferencia de un *namespace*, una clase es el elemento que permite englobar variables y métodos. Todo lo que esté escrito dentro de la clase, formará parte del objeto que más adelante se pueda crear y estará asociada a la clase. Como se ha comentado antes, una clase debe tener un nombre en *PascalCase*, y al menos una de las declaradas dentro del archivo debe tener el mismo nombre que este. En el ejemplo anterior se han declarado dos clases, llamadas *Program* y *OtherClass*, dentro de las cuales se empezaría a escribir todo el código a ejecutar. Para poder definir una clase se utiliza la palabra reservada *class* seguida del nombre, aunque puede tener algún otro modificador en el nombre como *internal* o *static*. No es obligatorio declarar una clase dentro de un *namespace*

3.2. Variables

Dentro de las clases es donde escribiremos el código que queremos que se ejecute. Uno de los elementos que encontraremos dentro de estas clases son las variables. Una variable es un dato que se asocia a un nombre y que se guarda en memoria para que pueda ser accedido o modificado a lo largo de la ejecución del programa. Para poder declarar una variable es necesario indicar el tipo de dato que puede guardar, siendo estos los posibles usos:

- *string*: utilizado para guardar palabras completas. La definición de este tipo de dato se debe poner entre comilla doble.
- *char*: utilizado para guardar letras. La definición de este tipo de dato se debe poner entre comilla simple.
- *bool*: utilizado para guardar valores de verdadero o falso (*true* / *false*). Muy útil para la toma de decisiones lógicas.
- *short*, *int*, *long*: utilizado para guardar números enteros. La diferencia entre ellas es la cantidad de números que puede guardar.
- *float*, *double*: utilizado para guardar números con decimales. La diferencia entre ellos es la precisión de la coma que puede guardar.

© Centro de Formación Profesional Europeo de Madrid

Para poder definir una variable se indica el tipo de esta, seguido del nombre e igualada al valor que se quiere almacenar. Aquí podemos ver algún ejemplo de definición de estas:

```
namespace EjemploNamespace
{
    public class Program
    {
        char letra = 'a';
        string nombre = "Daniel";
        short departamento = 1;
        int edad  = 24;
        long telefono = 34600000000;
        float altura = 1.74f;
        double pi = 3.14159523123123123123123123123;
    }
}
```

Una cosa que hay que tener en cuenta cuando se definen variables, es que su valor puede mutar siempre y cuando no se declaren como constantes. Esto quiere decir que se puede declarar una variable con un valor determinado y más adelante, en la ejecución del programa, reasignarle un valor. El único caso en el que no se puede dar esto es en las variables constantes, pudiendo declararlas de tal forma si se añade la palabra reservada *const* en la definición de esta:

```
const string nombreLegal = "NOMBRE INMUTABLE";
```

Importante

Si una variable se declara con un tipo y nombre, pero no se le da valor, este tomará el valor de NULL. Esto hay que tenerlo en cuenta a la hora de utilizar las variables ya que si se quiere acceder a un método de una variable cuyo valor es NULL obtendremos un error de NULLPOINTEREXCEPTION.

En C# existe un mecanismo para minimizar este tipo de error es y es el NULL SAFETY. Esto implica que cuando declaramos una variable cuyo valor es nulo pero podrá cambiar a lo largo del programa, se puede declarar con una ? y así indicar al compilador el posible NULL. Cuando se utilice esta variable, tendremos que indicar el posible valor con una ?

```
string? nombre = null;

Console.WriteLine(nombre?.Length);
```


© Centro de Formación Profesional Europeo de Madrid

Además de estos tipos de variables, también podemos diferenciar variables según su ámbito. Esto indica el lugar desde el cual pueden ser accedidas dichas variables y dependen de donde hayan sido creadas. Tenemos:

- Ámbito de clase: aquellas declaradas dentro de una clase y que pueden ser accedidas desde cualquier parte.
- Ámbito de bloque: aquellas declaradas dentro de un bloque (método, estructura de control, etc.) y que solo pueden ser accedidas desde donde han sido declaradas.

```
public class Program

{

    string variableClase = "variable de clase";


    static void Main(string[] args)

    {

        // declaración de variable con posibilidad de null de clase

        string? nombre;

        Console.WriteLine("Para comenzar por favor introduce tu nombre");

        nombre = Console.ReadLine();

        Console.WriteLine("Perfecto {0} ya podemos comenzar", nombre);

        if (true)

        {

            // declaración de una variable de bloque, solo accesible desde el if

            string accesible = "Variable no accesible desde fuera";

            Console.WriteLine (accesible);

        }


        // error ya que la variable no es accesible

        Console.WriteLine (accesible);

    }


    void metodo()

    {
```


© Centro de Formación Profesional Europeo de Madrid

```
        // acceso a una variable de clase
        Console.WriteLine (variableClase);


        // error ya que la variable no es accesible
        Console.WriteLine (nombre);
    }
}
```

3.3. Métodos

Si bien las variables vistas en el punto anterior representan espacios en memoria para poder guardar datos y que estos sean utilizados en cualquier parte del programa, los métodos representan las funciones que se pueden ejecutar desde la clase.

En C# existe un método especial llamado *Main*. Este se encarga de iniciar la ejecución del programa siempre y cuando se utilicen clases. Es importante si el programa cuenta con varias clases, solo una de ellas tenga declarado el método *Main*, ya que así es donde empezará la ejecución y desde ahí se llamará al resto de clases:

```
public class Program
{
    static void Main(string[] args)
    {
        // declaracion de variable con posibilidad de null
        string? nombre;
        // ejecución del programa
        Console.WriteLine("Comenzando la ejecución del programa");
        Console.WriteLine("Para comenzar por favor introduce tu nombre");
        nombre = Console.ReadLine();
        Console.WriteLine("Perfecto {0} ya podemos comenzar", nombre);
    }
}
```

Aunque a continuación se definirán cada uno de los elementos de los métodos, tan solo indicar que la palabra reservada *static* hace referencia a la posibilidad de acceder directamente a aquel elemento que le acompañe, en este caso el método *Main*.

© Centro de Formación Profesional Europeo de Madrid

Todos los métodos tienen una forma de crearse común (lo que se conoce como *firma*) y es la siguiente:

Modificador_acceso Modificador_retorno Nombre_Método (parámetros) { cuerpo }

Veamos que son cada una de las cosas:

- Modificador de acceso: indica cuál es el nivel de acceso que tendrá dicho método. Es decir, quién o desde dónde se podrá acceder al método. Este modificador podrá ser:
  - *Private*: nadie, excepto código de la misma clase donde está definido el método, podrá acceder a este.
  - *Protected*: tan solo podrán acceder al método los elementos ubicados dentro del mismo paquete y clases que han heredado de la clase donde se define el método.
  - *Public*: todos los objetos de la clase pueden acceder al método, aunque estén declarados en cualquier sitio.
- Modificar de retorno: indica el tipo de dato que devolverá el método cuando termine de realizar todas las ejecuciones indiadas en el cuerpo. En el caso de no querer retornar nada y que solo realice ejecuciones, se indicará el retorno de *void*.
- Nombre del método: utilizando notación *PascalCase* y con un nombre descriptivo para facilitar su uso.
- Parámetros: datos que el método necesita para poder ser llamado. Simplemente son referencias que se indicarán en el momento de llamar al método.
- Cuerpo: todas las líneas de código que se ejecutarán cuando el método sea llamado. Cuando en el tipo de retorno se indique algo diferente a *void*, la última línea del método tendrá que ser *return* acompañado del dato que retorna

Un ejemplo de la creación de métodos sería el siguiente:

```
public void realizarSuma(int operando1, int operando2){

    Console.WriteLine("El resultado de la suma es {0}",operando1*operando2);

}



public int realizarMultiplicacion(int operando1, int operando2){

    return operando1 * operando2;

}
```

Una vez creados los métodos, tan solo sería necesario llamarlos como realizarSuma(5,7)o realizarMultiplicacion(3,5). El primero de los métodos simplemente realiza la suma y saca un mensaje por consola mientras que la llamada al segundo método sería lo mismo que tener el número 15, para poder utilizarlo así en cualquier sitio.

© Centro de Formación Profesional Europeo de Madrid

Importante

En estas primeras unidades, todos los métodos tendrán la palabra reservada *static*, para así poder llamarlos desde el método *Main*. Cuando se explique la orientación a objetos esto cambiará

```
class Program
  {


    static public void Main(String[] args)
    {
       realizarSuma(1, 7);
       Console.WriteLine("El resultado de la multiplicación es {0}", realizarMultiplicacion(8, 2));
    }


    static public void realizarSuma(int op1, int op2)
    {
       Console.WriteLine("El resultado de la suma es {0}", op1 + op2);
    }


    static public int realizarMultiplicacion(int op1, int op2)
    {
       return op1 * op2;
    }
  }
```

La salida del programa será:

El resultado de la suma es 8

El resultado de la multiplicación es 16

© Centro de Formación Profesional Europeo de Madrid

4. Operadores

Los dos elementos vistos anteriormente permiten tanto gestionar los datos y poder guardarlos como agregar funcionalidad a una clase para que esta pueda ser ejecutada. Al mismo tiempo, se ha visto que dentro de los métodos existe lo que se conoce como cuerpo, donde se escriben todas las líneas de código que se deben ejecutar. Estas líneas pueden ser desde definición de variables, gestión de datos, estructuras de control para poder controlar el flujo del problema, etc., y en todos estos casos intervienen lo que se conoce como operadores, que no son más que lo que su nombre indica: elementos que permiten realizar operaciones de diferente tipo con valores. Los tipos de operadores existentes son:

- Aritméticos: aquellos que realizan operaciones aritméticas entre operandos de tipo numérico.

Unarios	

variable++	`int valor = 0;`
	`// la variable pasa a tomar el valor de 1`
	`valor++;`

variable--	`// la variable pasa a tomar el valor de 0` `valor--;`

-variable	`valor = 0;`
	`valor++;`
	`// la variable pasa a tomar el valor de -1`
	`valor = -valor;`

+variable	`valor = 0;`
	`+valor;`
	`// la variable pasa a tomar el valor de +0`

Binarios	
-	Resta aritmética entre dos operadores.
+	Suma aritmética entre dos operadores.
*	Multiplicación aritmética entre dos operadores.
/	División aritmética entre dos operadores. En este caso hay que tener en cuenta que la división entre dos números enteros es un número entero.

© Centro de Formación Profesional Europeo de Madrid

%	Resto aritmético entre dos operadores. Se trata del resto de la división entre dos números.

Asignaciones compuestas	Se trata de aplicar la operación al mismo tiempo que se hace una asignación. Para ello se utiliza el operando seguido del signo =
	`int numero = 4;`
	`// sería lo mismo que aplciar numero = 4 + 10`
	`numero += 10;`
	`// numero = 14`

- Comparación: aquellos que permiten la comparación entre dos valores.

Operadores comparación	
>	Obtiene como resultado *true* si el operando de la izquierda es mayor que el operando de la derecha. En caso contrario *false*.
>=	Obtiene como resultado *true* si el operando de la izquierda es mayor o igual que el operando de la derecha. En caso contrario *false*.
<	Obtiene como resultado *true* si el operando de la izquierda es menor que el operando de la derecha. En caso contrario *false*.
<=	Obtiene como resultado *true* si el operando de la izquierda es menor o igual que el operando de la derecha. En caso contrario *false*.
==	Obtiene como resultado *true* ambos operandos son iguales. En caso contrario *false*.
!=	Obtiene como resultado *true* si ambos operandos son diferentes. En caso contrario *false*.

Un ejemplo de uso de cada uno de los operadores sería el siguiente:

```
int numeroUno = 10;

int numeroDos = 6;

bool relacionUno = numeroUno>numeroDos; // 10>6 -> true

numeroDos=10;

bool relacionDos = numeroUno >= numeroDos; // 10>10 ->true

numeroDos = 17;

bool relacionTres = numeroDos<numeroUno; // 17<10 -> false

bool relacionCuatro = numeroUno<=numeroDos; // 10<=17 -> true

bool relacionCinco = numeroDos == numeroUno; // 17 == 10 -> false
```


© Centro de Formación Profesional Europeo de Madrid

```
numeroDos=10;

bool relacionSeis = numeroUno!=numeroDos; // 10 != 10 -> false
```

- Lógicos: permite agrupar varias sentencias lógicas o condiciones para obtener un resultado único. Estos operadores se basan en las puestas lógicas AND y OR

Operadores lógicos	
&&	Obtiene como resultado *true* todas las condiciones de la sentencia son verdaderas. En caso contrario el resultado es *false*. A diferencia del operador & es que si la primera condición es *false* el resto no se evalúa.
||	Obtiene como resultado *true* si una de las condiciones de la sentencia es verdaderas. En caso contrario el resultado es *false*. A diferencia del operador & es que si la primera condición es *true* el resto no se evalúa.
&	Obtiene como resultado *true* si el operando de la izquierda es menor que el operando de la derecha. En caso contrario *false*.
|	Obtiene como resultado *true* si el operando de la izquierda es menor o igual que el operando de la derecha. En caso contrario *false*.

Un ejemplo de uso de los operadores sería el siguiente:

```
int numeroUno = 10;

int numeroDos = 6;

int numeroTres =30;

int numeroCuatro = 26;

bool relacionUno = numeroUno>0 && numeroDos<numeroTres; // 10>0 && 6<30 -> true

bool relacionDos = numeroDos<0 || numeroDos<numeroCuatro; // 6<0 || 6<26 -> true

bool relacionTres =  numeroCuatro> 2*numeroUno && !(numeroTres>numeroCuatro)&& !false; // !(30>26) && 26>20

bool relacionCuatro =  ( (numeroCuatro> 2*numeroUno  || numeroCuatro> 2*numeroUno) && (numeroTres>numeroCuatro || false)) || true; // !(30>26) && 26>20
```


© Centro de Formación Profesional Europeo de Madrid

- *Bit* a *bit* y desplazamiento: aquellos que permiten el movimiento de *bits* entre dos operandos. En este caso, no se verán estos operadores ya que su utilidad es avanzada.

Ejercicio

Realiza un programa que permita realizar las siguientes acciones:

- Pedir al usuario 2 números y guardarlos en variables.
- Crear 5 métodos para la realización de todas las operaciones aritméticas explicadas. Éstos tendrán dos parámetros de entrada (los operadores a utilizar) y como retorno el resultado de la operación.
- Llama desde el método *main* a todos los métodos creados pasando como parámetros los datos pedidos al usuario y muestra su resultado.
- Comenta cada uno de los métodos dentro del código, indicando su funcionalidad y sus parámetros.

Resumen

Los principales elementos dentro de un programa informático son las clases, las variables y los métodos. Los primeros (junto con los *namespace*) sirven para poder organizar el código en diferentes "estructuras" que engloban funcionalidades (más adelante se verá que estas estructuras se conocen como objetos). Las variables son datos (que pueden ser de diferentes tipos) que guardan una información para que pueda ser modificada o accedida en cualquier momento en la ejecución del programa. Por último, los métodos son las estructuras que realizan las funcionalidades y pueden ser llamadas tantas veces cono sea necesario.

Para poder dotar a todo esto de una funcionalidad completa, existe lo que se conoce con el nombre de operadores, existiendo diferentes tipos (aritméticos, asignación, lógicos y comparación), los cuales permiten operar con los datos para poder realizar tantas transformaciones y gestiones con los datos como sean necesarias.

Referencias bibliográficas

Métodos C#. <https://learn.microsoft.com/es-es/dotnet/csharp/programming-guide/classes-and-structs/methods>.

Operadores C# <https://learn.microsoft.com/es-es/dotnet/csharp/language-reference/operators/>.

© Centro de Formación Profesional Europeo de Madrid

Universidad Europea Online
CENTRO PROFESIONAL

Todos los derechos de propiedad intelectual de esta obra pertenecen en exclusiva al © Centro de Formación Profesional Europeo de Madrid. Queda terminantemente prohibida la reproducción, puesta a disposición del público y en general cualquier otra forma de explotación de toda o parte de la misma.

La utilización no autorizada de esta obra, así como los perjuicios ocasionados en los derechos de propiedad intelectual e industrial del © Centro de Formación Profesional Europeo de Madrid, darán lugar al ejercicio de las acciones que legalmente le correspondan y, en su caso, a las responsabilidades que de dicho ejercicio se deriven.

Ve más allá